

INSTITUTO FEDERAL

Sergipe

Campus Tobias Barreto

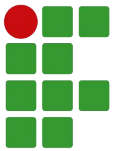
Programação 1

JavaScript

Prof. Christiano Lima Santos



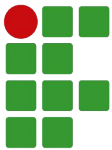
COORDENAÇÃO DE INFORMÁTICA



Objetivos

Espera-se que cada aprendiz alcance os seguintes objetivos:

- ❑ Compreender a lógica de programação e seus principais conceitos (variáveis, operações, atribuições, condições, repetições, vetores e funções);
- ❑ Compreender os comandos/instruções básicas na linguagem de programação JavaScript;
- ❑ Ser capaz de criar algoritmos/programas para tarefas simples.

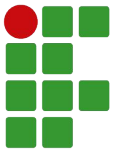


Conteúdo do curso

- ❑ Introdução à Programação
- ❑ Introdução a JavaScript
- ❑ Identificadores, tipos de dados, variáveis e constantes
- ❑ Operadores e expressões
- ❑ Comentários, funções para conversão e template strings
- ❑ Comandos condicionais
- ❑ Comandos de repetição
- ❑ Funções
- ❑ Objetos e Arrays
- ❑ Date, Math e String

Introdução à Programação

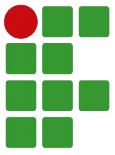
Parte 1



Sumário

— — —

- ❑ Introdução à Programação
- ❑ Introdução aos algoritmos
- ❑ O que é uma linguagem de programação
- ❑ Ambiente de Desenvolvimento Integrado (IDE)

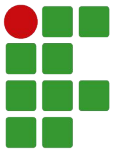


Introdução à Programação

— — —

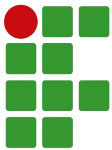
- ❑ Do ENIAC ao microcomputador, dos supercomputadores à computação ubíqua, a Computação passou por grandes evoluções e transformações;

- ❑ Quais as contribuições da Computação nas seguintes áreas?
 - ❑ Educação;
 - ❑ Saúde;
 - ❑ Finanças;
 - ❑ Política.



Introdução à Programação

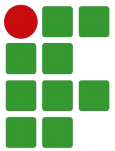
- ❑ Você consegue imaginar um mundo sem a Computação?
- ❑ De forma similar, o mundo não seria o mesmo sem os avanços da programação dos computadores!



Mas... Programar é difícil?

- ❑ Possui os mesmos desafios de aprender algo muito diferente pela primeira vez!
 - ❑ Falar;
 - ❑ Ler e escrever;
 - ❑ Cálculos aritméticos.

- ❑ Aprender a programar exige praticar!

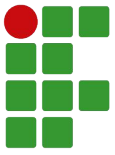


Sugestão de vídeo: Dicas para aprender a programar

— — —

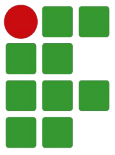


Endereço: <https://youtu.be/ZtMzB5CoekE>



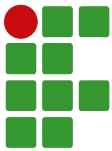
O que são algoritmos?

- ❑ São uma forma de descrever como uma determinada tarefa deve ser executada, de forma clara e objetiva;
- ❑ Podem ser usados para descrever como um *software* deve proceder para cumprir uma tarefa.



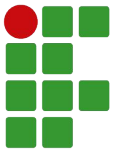
Características de um algoritmo

- ❑ **Finitude** – Deve ser formado por um conjunto finito de passos, isto é, possuir início e fim;
- ❑ **Definição** – Cada ação ou passo de um algoritmo deve ser escrito de forma bastante precisa, sem ambiguidades ou dúvidas quanto à sua execução;
- ❑ **Sequencial** – Um algoritmo é formado por uma série de passos que devem ser executados em uma ordem sequencial;



Características de um algoritmo (cont.)

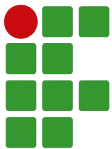
- ❑ **Entradas** – Deve apresentar zero ou mais entradas, isto é, informações a serem inseridas por um usuário ou por outra fonte externa;
- ❑ **Saídas** – Deve apresentar uma ou mais saídas, isto é, informações pré-definidas e/ou resultantes do processamento das entradas;
- ❑ **Eficiência** – Suas operações devem ser básicas o suficiente tal que um ser humano seja hábil a executá-las com precisão em papel e lápis em um tempo finito.



Formas de representação dos algoritmos

— — —

- ❏ Descrição narrativa
- ❏ Fluxograma
- ❏ Pseudocódigo
- ❏ Em uma linguagem de programação



Descrição narrativa

- ❑ Descreve como uma tarefa deve ser executada por meio de um texto, seja ele verbal ou escrito;

Exemplo - Receita de bolo

Separe os ingredientes

Bata os ovos em ponto de neve na batedeira

Acrescente açúcar e farinha de trigo

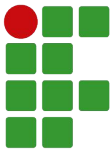
Acrescente uma colher de manteiga

Acrescente uma colher de fermento em pó

Coloque na forma

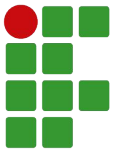
Coloque no forno e asse

Tire da forma e sirva



Descrição narrativa (cont.)

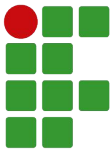
- ❑ **Vantagem:** muito fácil de ser compreendida e empregada no dia-a-dia, qualquer um capaz de compreender alguma língua pode falar ou escrever uma narrativa na mesma;
- ❑ **Desvantagem:** inexistência de regras que ditem como cada ação deve ser descrita a fim de evitar ambiguidades ou problemas de clareza quanto à sua compreensão.



Fluxograma

Trata-se de um diagrama composto por várias ações e entidades representadas em “caixas” de diversos formatos (cada formato determina o tipo de ação a ser tomada ali) e setas interligando todas essas caixas de forma a criar um “caminho” com início e fim.

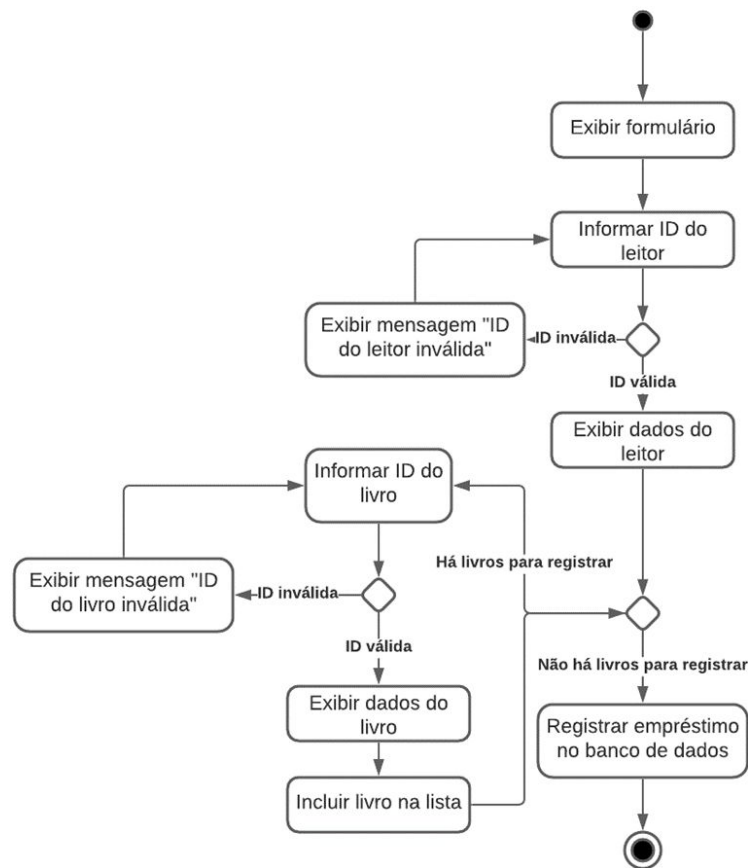
A sequência de ações que deve ser tomada, então, é encontrada “percorrendo-se” o fluxograma de seu início até o seu fim, sendo então executada cada ação ou teste como está escrito.

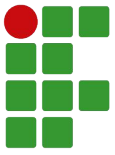


Fluxograma (cont.)

— — —

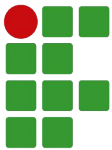
Exemplo - Efetuar empréstimo de livro





Fluxograma (cont.)

- ❑ **Vantagem:** é mais concisa que a descrição narrativa, representando visualmente desvios e laços;
- ❑ **Desvantagem:** necessita o aprendizado do processo de construção de fluxogramas.

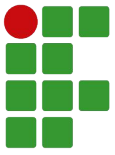


Pseudocódigo

- ❑ É a representação textual das ações por meio de comandos pré-definidos em algo parecido com uma linguagem de programação, porém muito mais próximo da nossa;
- ❑ No Brasil, temos o Portugol ou Português Estruturado.

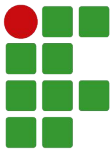
Exemplo - Cálculo da média de um aluno

```
programa calculo_de_media;  
  
leia notaA, notaB, notaC;  
  
media <- (notaA + notaB + notaC) / 3;  
  
imprima media;
```



Pseudocódigo (cont.)

- ❑ **Vantagem:** sua transcrição para uma linguagem de programação é bastante direta;
- ❑ **Desvantagem:** requer aprender todas as regras daquele pseudocódigo, o que pode ser quase tão complexo quanto aprender uma linguagem de programação.

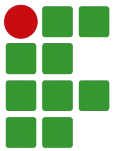


Em uma linguagem de programação

- ❑ Pode-se utilizar uma linguagem de programação tal que possamos elaborar diretamente o código-fonte do programa que desejamos.

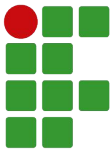
Exemplo - Cálculo da média de um aluno

```
let notaA = 7;  
let notaB = 8;  
let notaC = 9;  
let media = (notaA + notaB + notaC)/3;  
alert("Média: " + media);
```



Em uma linguagem de programação (cont.)

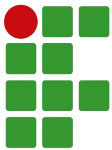
- ❑ **Vantagem:** por meio dela, já temos o código-fonte do programa que desejamos construir, assim, precisamos somente compilar ou interpretar;
- ❑ **Desvantagem:** é necessário conhecimentos suficientes de uma linguagem de programação a fim de escrever o código-fonte da mesma, exigindo assim muito mais do aprendiz.



Elementos fundamentais de um algoritmo

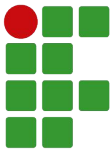
- ❑ Todo algoritmo possui:
 - ❑ Entrada;
 - ❑ Processamento;
 - ❑ Saída.
- ❑ Para ficar mais claro, consideremos como exemplo um algoritmo que, dado um número N , deve retornar o fatorial de N como solução.





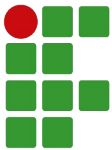
Elementos fundamentais de um algoritmo (cont.)

- ❑ **Entrada:** são os dados de entrada do algoritmo, informação necessária para a execução correta do mesmo e deve ser informada pelo usuário do sistema e/ou vir de outras fontes externas (como bancos de dados, arquivos etc.);
- ❑ No nosso exemplo, o valor de N será a nossa entrada, necessária para o cálculo.



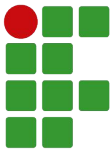
Elementos fundamentais de um algoritmo (cont.)

- ❑ **Processamento:** são os procedimentos utilizados para, a partir dos dados de entrada, chegar ao resultado final;
- ❑ Em nosso exemplo, todo o cálculo necessário para determinar o fatorial de N deve ser considerado como parte do processamento.



Elementos fundamentais de um algoritmo (cont.)

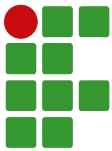
- ❑ **Saída:** são os dados já processados e prontos para serem enviados ao usuário ou armazenados em alguma fonte externa;
- ❑ Em nosso exemplo, o resultado final, isto é, o fatorial de N , é a nossa saída.



O que é uma linguagem de programação?

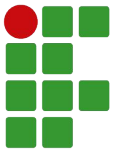
- ❑ Assim como estabelecemos uma linguagem para a comunicação entre duas ou mais pessoas, os computadores possuem sua própria linguagem;
 - ❑ Linguagem de máquina (ou linguagem de baixo nível);

- ❑ Entretanto, por ser uma linguagem difícil para nós entendermos, foram criadas linguagens de programação mais parecidas com a linguagem humana, como C/C++, Java, PHP, JavaScript etc.
 - ❑ Linguagens de alto nível.



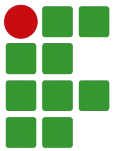
O que é uma linguagem de programação? (cont.)

- ❑ Assim, uma linguagem de programação refere-se ao conjunto de regras sintáticas e semânticas utilizado para definir um programa de computador;
- ❑ Por meio dela, somos capazes de especificar os comandos necessários para a execução de tarefas no computador. Essa especificação é geralmente conhecida como código-fonte.



O que é uma linguagem de programação? (cont.)

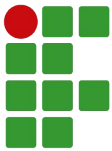
- ❑ Então, um *software* é responsável por converter o nosso código-fonte (linguagem de alto nível) em linguagem de máquina;
 - ❑ Compilação;
 - ❑ Interpretação;
 - ❑ Compilação para Máquinas Virtuais.



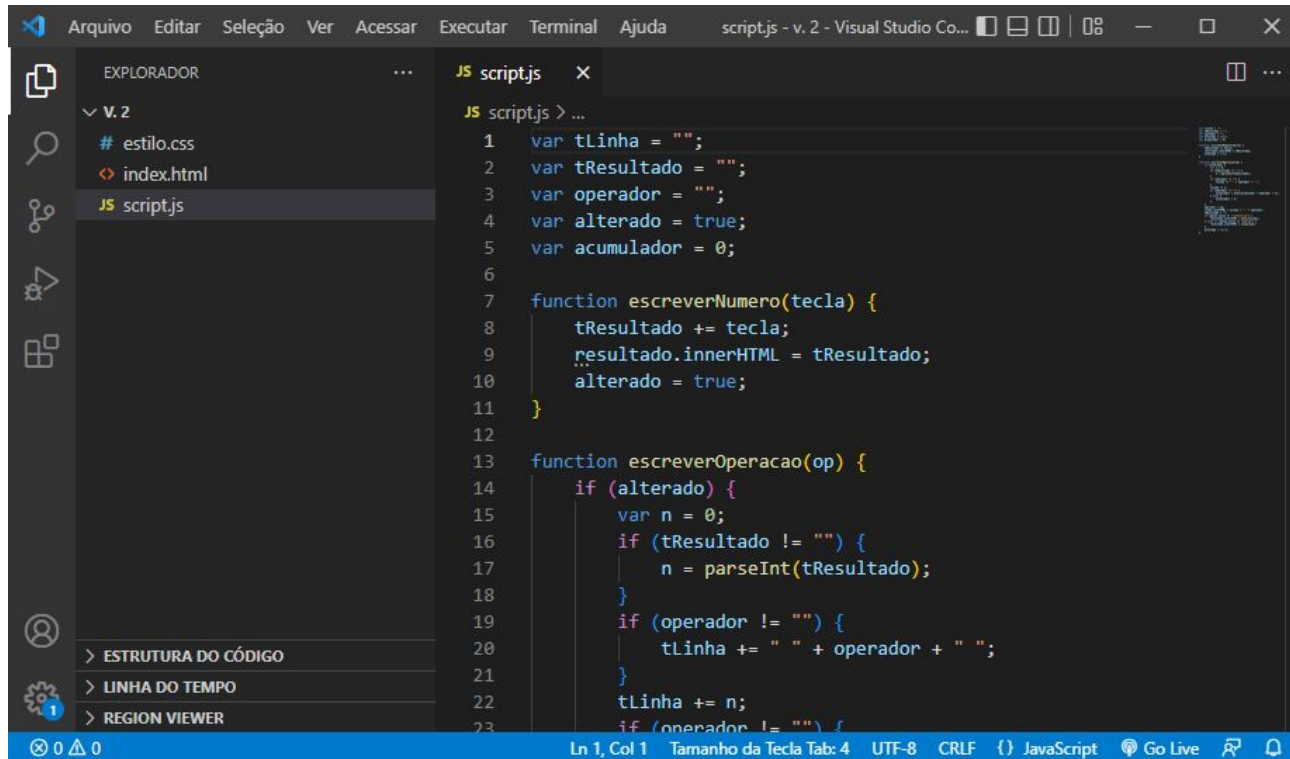
Ambiente de Desenvolvimento Integrado ou Integrated Development Environment (IDE)

- ❑ Refere-se a um ambiente integrado onde o desenvolvedor pode organizar, editar, compilar e testar os arquivos relacionados ao seu programa, de forma fácil e rápida.

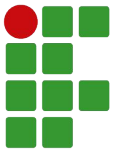
Curiosidade: Antes do uso de IDEs, o código-fonte era escrito em qualquer editor de texto (sem benefícios como funcionalidades para “autocompletar código”, verificação de sintaxe em tempo de edição e outras) e era comum o uso de prompt de linha de comando para as tarefas de compilação, linkedição etc.



Ambiente de Desenvolvimento Integrado ou Integrated Development Environment (IDE) (cont.)



IDE do Visual Studio Code

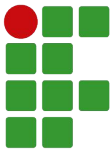


Ambiente de Desenvolvimento Integrado ou Integrated Development Environment (IDE) (cont.)

— — —

- ❏ Em nossa disciplina, estudaremos a linguagem de programação JavaScript utilizando a IDE do Visual Studio Code (VSCode):

<https://code.visualstudio.com>

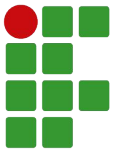


Exercício

1. Por meio de uma descrição narrativa, escreva um algoritmo para a troca de um pneu furado no meio da estrada;
2. Agora, descreva um algoritmo para o processo de matrícula no IFS;
3. Por fim, descreva um algoritmo para calcular a média de um aluno a partir de suas três notas.
Dica: você pode receber dados com comandos como “Leia nome” ou “Leia nota” e exibir o resultado com comandos como “Imprima nome” ou “Imprima nota”.

Introdução a JavaScript

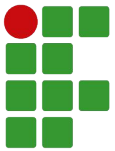
Parte 2



Sumário

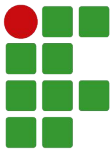
— — —

- ❏ O que é JavaScript?
- ❏ História do JavaScript
- ❏ Editores para JavaScript
- ❏ Como usar JavaScript em uma página web
- ❏ Saiba mais



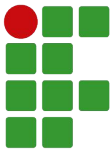
Recordando...

- ❑ O que é programar?
- ❑ O que é um algoritmo?
- ❑ O que é uma linguagem de programação?



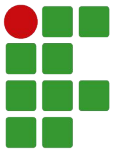
O que é JavaScript?

- ❑ **Linguagem de programação** interpretada com tipagem dinâmica (linguagem de script);
 - ❑ Linguagem de programação – permite a criação de rotinas (conjuntos de instruções) com finalidades específicas. Ex: validar entradas em um formulário, alterar textos, tags ou propriedades CSS de uma página etc.



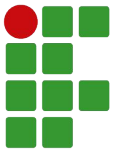
O que é JavaScript? (cont.)

- ❑ Linguagem de programação **interpretada** com tipagem dinâmica (linguagem de script);
 - ❑ Interpretada – não é compilada, isto é, o código escrito pelo desenvolvedor é lido e executado pelo interpretador (neste caso, o navegador).



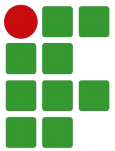
O que é JavaScript? (cont.)

- ❑ Linguagem de programação interpretada **com tipagem dinâmica** (linguagem de script);
 - ❑ Tipagem dinâmica - variáveis podem receber dados de qualquer tipo.



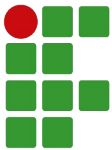
História do JavaScript

- ❑ Antes do surgimento das linguagens de script para web, as páginas eram geralmente estáticas e ofereciam poucas formas de interação, limitando-se a hyperlinks e formulários.



História do JavaScript

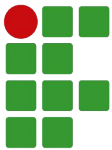
- ❑ Netscape (líder em navegadores) lança em setembro de 1995 o navegador Netscape 2.0 com suporte a uma nova linguagem, LiveScript;
- ❑ Em dezembro de 1995, em anúncio conjunto com a Sun Microsystems, muda o nome para JavaScript e adiciona suporte à tecnologia Java em seu navegador (applets);
 - ❑ Estratégia de marketing!



História do JavaScript

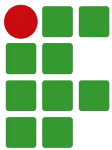
- ❑ Novembro de 1996 - Netscape é submetida e aprovada como padrão industrial, resultando na versão padronizada ECMAScript / ECMA-262;

- ❑ Atualmente na versão ECMAScript 2022, JavaScript é um padrão em programação, tendo seu uso aliado a outras tecnologias:
 - ❑ jQuery, JSON e Ajax
 - ❑ NodeJS
 - ❑ React



Editores para JavaScript

- ❑ Inicialmente, usaremos nosso código JavaScript dentro de páginas HTML;
 - ❑ Por isso, salvaremos nossos arquivos com a extensão “.html”;
- ❑ Pode-se escrever código JavaScript em qualquer editor de texto...
 - ❑ Bloco de Notas, Notepad++, Brackets, VSCode etc.
- ❑ ...e ver o resultado produzido em qualquer navegador;
 - ❑ Google Chrome, Mozilla Firefox, Microsoft Edge, Safari etc.



Como usar JavaScript em uma página web

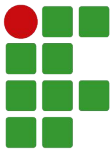
— — —

- ❑ Há três maneiras diferentes de utilizar código JS dentro de uma página web;
- ❑ A primeira que vamos aprender é inserindo dentro de uma tag `<script>`;
- ❑ A função `alert()` é uma das formas de exibir o resultado de um código JS.¹

exemplo-2.1.html

```
<script>  
    alert('Olá, mundo');  
</script>
```

¹Saiba mais em: https://www.w3schools.com/js/js_output.asp



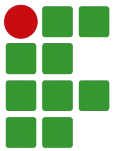
Verificando erros nos scripts

— — —

- ❑ Os navegadores implementam ferramentas que podem ser usadas para identificar erros em scripts bem como para testar funcionalidades;
- ❑ No Google Chrome e Mozilla Firefox, por exemplo, há o “Console”, disponível em “Ferramentas do Desenvolvedor” (tecla de atalho: F12).

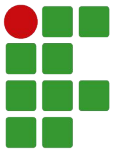
exemplo-2.2.html

```
<script>  
    alert(Olá, mundo);  
</script>
```



Exercício

1. Instale o Visual Studio Code em seu computador;
2. Crie em seu computador uma pasta para guardar os exemplos e exercícios de Programação 1 e abra-a no VSCode (botão direito sobre a pasta, opção “Abrir com Code”);
3. No VSCode, instale a extensão “Live Server” (de Ritwick Dey);

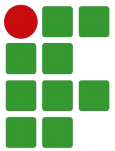


Exercício (cont.)

4. Ainda no VSCode, recrie o exemplo 2.2, salve e execute-o no navegador (botão direito sobre o arquivo, opção “Open with Live Server”);
5. No navegador, abra as Ferramentas do Desenvolvedor (tecla F12), encontre a opção do Console e identifique a mensagem de erro;
6. Agora, retorne ao VSCode, corrija o código, salve e execute-o novamente no navegador. Agora ele deveria funcionar!

Identificadores, Tipos de Dados, Variáveis e Constantes

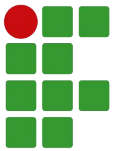
Parte 3



Sumário

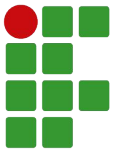
— — —

- ❏ Identificadores
- ❏ Tipos de dados
- ❏ Variáveis
- ❏ Constantes
- ❏ Recebendo entrada de dados do usuário
- ❏ Palavras reservadas



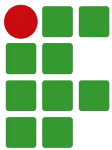
Recordando...

- ❑ O que é o Visual Studio Code?
- ❑ Qual linguagem de programação estudaremos?
- ❑ Para que serve a tag `<script>`?



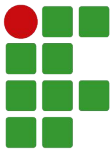
Identificadores

- ❑ Um identificador refere-se ao nome dado a uma variável, tipo, função, constante etc.
- ❑ JavaScript é uma linguagem *case-sensitive*, isto é, que diferencia letras maiúsculas de minúsculas, então Objeto, objeto e OBJETO são identificadores diferentes e podem se referir a entidades (variáveis, constantes, funções etc.) diferentes dentro do código.



Regras para criação de identificadores

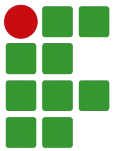
- ❑ Um identificador deve conter somente letras (maiúsculas ou minúsculas), dígitos ou um grupo restrito de caracteres especiais (`_`, `$`);
 - ❑ **Obs:** Em JavaScript, não é muito comum criarmos variáveis iniciando com “`_`”, muito menos com “`$`” (geralmente usados em *frameworks*, como o jQuery);
- ❑ Não podem ser utilizados outros caracteres especiais, como letras com acento, “`&`” ou espaço;
- ❑ O primeiro caractere de um identificador não pode ser um número;



Regras para criação de identificadores (cont.)

- ❑ Letras maiúsculas e minúsculas podem ser utilizadas, mas é uma boa prática de programação em nomes de variáveis e funções utilizar sempre letras minúsculas, excetuando a primeira letra da segunda palavra em diante que compõe o identificador.

Exemplos: contador, somaTotal, pagamentoDosEmpregados.



Exemplos de identificadores

— — —

Identificadores válidos

a

A

_a

\$usuario

a_1

casa1

somaTotal

Identificadores inválidos

1a

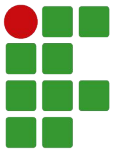
casa 1

calçada

soma total

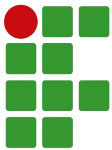
Salário Base

3nota



Tipos de dados

- ❑ Um tipo especifica as características de uma informação, determinando assim quais valores podem ser armazenados, como podem ser lidos ou escritos e quais as operações possíveis com a mesma. Por exemplo, há operações que podem ser feitas sobre tipos numéricos, outras aplicáveis a textos;
- ❑ O tipo determina também o intervalo de valores possíveis. Por exemplo, existem tipos em que “cabem” números maiores do que em outros.

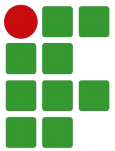


Tipos de dados mais comuns em JavaScript

— — —

Tipo	Valores aceitos	Exemplos de valores
int	números inteiros	10 -8
float	números decimais	10.0 -8.3
boolean	valores lógicos	true (verdadeiro) false (falso)
string	textos	"moeda" "Santa Ceia" ""
null	nulo (<i>indica que há nenhum valor ali</i>)	null

- JavaScript apresenta tipagem fraca, não sendo necessário declarar o tipo de uma variável e a mesma poderá receber valores de um tipo diferente, posteriormente.

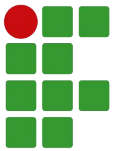


Tipos de dados mais comuns em JavaScript

- ❑ Nesse exemplo, a variável `x` é declarada inicialmente com um valor numérico e, sucessivamente, atribui-se a ela uma string, um boolean e, por fim, `null`.

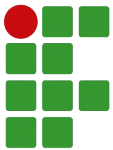
Exemplo

```
var x = 0;  
x = "Olá!";  
x = true;  
x = null;
```



Variáveis

- ❑ Trata-se de um local na memória reservado para armazenar valores de um determinado tipo. Esse local na memória poderá ser referenciado (isto é, acessado para leitura ou escrita) a partir do identificador dado àquela variável;
- ❑ Vale lembrar que o valor armazenado por uma variável **pode ser alterado durante a execução** do programa, a principal distinção entre uma variável e uma constante!

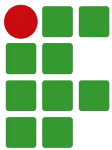


Declaração de variáveis¹

- ❏ Em JavaScript, nós precisamos declarar uma variável antes de usá-la (isto é, acessar seu conteúdo). Isso pode ser feito de três maneiras:

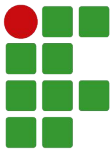
Tipo de Declaração	Exemplo	Observação
Usando “var”	var x = 10;	Funciona desde 1995 Escopo local
Usando “let”	let x = 10;	Funciona desde 2015 Escopo local
Usando nada	x = 10;	Funciona desde 1995 Escopo global

¹Saiba mais em: https://www.w3schools.com/js/js_variables.asp



Declaração de variáveis (cont.)

- ❑ Em nossa disciplina, vamos declarar sempre as variáveis com “var” ou “let” (a terceira forma pode ser muito confusa durante a manutenção de um sistema);
 - ❑ Se você quiser garantir compatibilidade com navegadores de antes de 2015, use somente “var”;
 - ❑ Caso contrário, melhor usar somente o “let”;
- ❑ Variáveis globais serão declaradas no início do script.



Exemplos de declaração de variáveis

Declarações válidas

```
var a = 5;
```

```
let media = 6.5;
```

```
let i, j;
```

```
var nome = "Carlos";
```

```
var escola = 'IFS';
```

```
let ativo = false;
```

```
var nota1 = 5, nota2 = 7;
```

Declarações inválidas

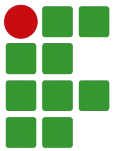
```
float b = 3;
```

```
var nota = 6,4;
```

```
var x y;
```

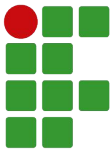
```
let escola = IFS;
```

```
let presente = TRUE;
```



Constantes

- ❑ Uma constante trata-se de uma referência (nome) para um dado valor, permitindo sua leitura (mas não escrita) a qualquer momento;
- ❑ É importante salientar que uma constante é, então, um valor (referenciado por um identificador) que **não pode ser alterado durante a execução** do programa!



Declaração de constantes

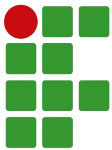
- ❑ Em JavaScript, nós precisamos declarar uma constante com a palavra “const” antes de usá-la.

Exemplos

```
const x = 8;
```

```
const nome = "Carlos", idade = 19;
```

Obs: Constantes foram introduzidas no JavaScript somente em 2015 (ECMA 2015).

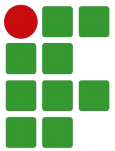


Recebendo entrada de dados (valores) do usuário

- ❑ Em JavaScript, podemos receber informações do usuário de diversas formas. Uma das mais simples é a função *prompt()*, que permite exibir uma mensagem para o usuário e devolve o valor digitado pelo mesmo como string (texto).

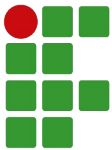
Exemplo

```
var nome = prompt("Qual é o seu nome?");  
alert("Olá, " + nome);
```

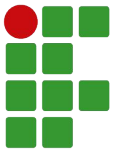
Palavras reservadas

- ❏ São palavras que possuem finalidades específicas dentro de uma linguagem de programação e, portanto, não podem ser usadas para outra finalidade (por exemplo, para declarar uma variável ou constante);



Palavras reservadas

- ❑ Alguns exemplos de palavras reservadas em JavaScript:
 - ❑ Instruções declarativas - `var`, `let`, `const`, `function` etc.
 - ❑ Comandos condicionais ou de repetição - `if`, `else`, `switch`, `case`, `for`, `while` etc.
 - ❑ E outras.
- ❑ **Obs:** Apesar de nomes de tipos de dados (`int`, `float`, `char`, `boolean` etc.) não serem palavras reservadas em JavaScript, não recomendamos seu uso na declaração de variáveis, constantes ou funções!

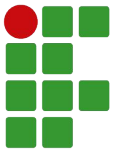


Exercício

1. Crie uma página com um script declarando as duas variáveis abaixo:
 - nome - "Christiano";
 - idade - 37.
2. Ao final, exiba cada uma das variáveis em um *alert()*;
3. Agora, altere o código para que o usuário informe seu nome e sua idade.

Operadores e Expressões

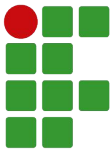
Parte 4



Sumário

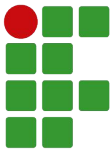
— — —

- ❏ Operadores, Operandos e Expressões
- ❏ Tipos de Operadores em JavaScript



Recordando...

- ❑ Quais os tipos de dados mais comuns em JavaScript?
- ❑ O que são variáveis?
- ❑ Como você declararia três variáveis para guardar as notas 5, 7.5 e 8.4?
- ❑ O que são constantes?
- ❑ Qual função usamos para receber dados do usuário?



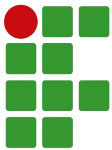
Operadores, Operandos e Expressões

- ❑ Um **operador** é um símbolo utilizado para identificar que uma determinada operação deve ser realizada sobre um ou mais parâmetros, denominados **operandos**;

Exemplos

Em “3 + 4”, o “+” é o operador, atuando sobre os operandos “3” e “4”.

Em “x + (3 * 5)”, temos o operador “+” atuando sobre os operandos “x” e “3 * 5”, e o operador “*” atuando sobre os operandos “3” e “5”.



Operadores, Operandos e Expressões (cont.)

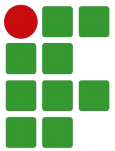
- ☐ Já uma **expressão** pode ser considerada um conjunto de operações efetuadas sobre certos dados a fim de obter um resultado;

Exemplos

“ $3 + 4$ ” é um exemplo de expressão;

“ $x * 5 / y$ ”, também é um exemplo de expressão;

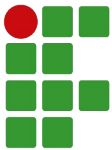
“ $7 > x$ ”, também!



Tipos de Operadores

— — —

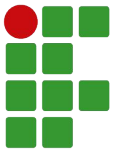
- ❑ Operadores aritméticos;
- ❑ Operadores relacionais;
- ❑ Operadores lógicos;
- ❑ Operadores para strings;
- ❑ Operadores de atribuição;
- ❑ Operadores especiais.



Operadores Aritméticos

— — —

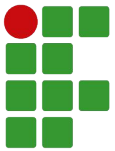
Operador	O que faz	Exemplo
+	Soma	$5 + 2$
++	Incrementa uma unidade	$i++$
-	Subtrai ou inverte o sinal	$5 - 2$ -3
--	Decrementa uma unidade	$i--$
*	Multiplica	$5 * 2$
/	Divide	$5 / 2$
%	Calcula o módulo (resto)	$5 \% 2$



Operadores Relacionais

— — —

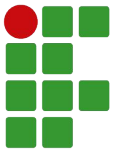
Operador	O que faz	Exemplo
>	É maior que...	$5 > 2$
>=	É maior ou igual a...	$5 >= 2$
<	É menor que...	$5 < 2$
<=	É menor ou igual a...	$5 <= 2$
==	É igual a...	$5 == 5$ $5 == "5"$
===	É igual e do mesmo tipo...	$5 === "5"$
!=	É diferente...	$5 != 2$



Operadores Lógicos

— — —

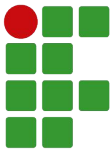
Operador	O que faz	Exemplo
&&	E	x && y
	Ou	x y
!	Não	!x



Operadores para Strings

— — —

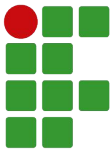
Operador	O que faz	Exemplo
+	Concatenação	$x + y$



Operadores de Atribuição

— — —

Operador	O que faz	Exemplo
=	Recebe (armazena um valor)	x = 3
+=	Recebe o valor dele adicionado com	x += 3
-=	Recebe o valor dele subtraído de	x -= 3
*=	Recebe o valor dele multiplicado por	x *= 3
/=	Recebe o valor dele dividido por	x /= 3
%=	Recebe o resto da divisão dele por	x %= 3

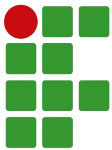


Operadores Especiais

— — —

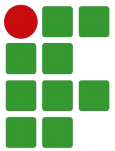
Operador	O que faz	Exemplo
? :	Segundo uma condição, retorna um valor ou outro	<code>a ? b : c</code>
in	Verifica se a propriedade especificada está no objeto passado	<code>title in pIntroducao</code>
new	Cria um objeto a partir de um construtor	<code>new String("teste")</code>

Obs: Detalharemos o uso desses operadores mais à frente.



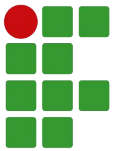
Exercício

1. Declare duas variáveis **a** (valor inicial 5) e **b** (valor inicial 7). A seguir, declare uma variável **c** e atribua a ela o resultado da multiplicação de **a** por **b**;
2. Agora, altere o código do exercício anterior, tal que o usuário informe (usando *prompt()*) os valores de **a** e **b** e mostre o resultado com um *alert()*.



Exercício (cont.)

3. Declare duas variáveis **x** (inicialmente 3.5) e **y** (inicialmente 5 vezes o valor de x). Declare, então, uma variável **z** e atribua a esta a metade do quadrado da soma de x e y;



Exercício

4. Agora, escreva as seguintes atribuições em JavaScript:

$$a = 1.0$$

$$b = -1.0$$

$$c = -12.0$$

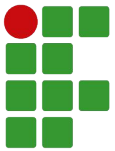
$$\text{delta} = b^2 - 4ac$$

$$x1 = \frac{-b - \sqrt{\text{delta}}}{2a}$$

$$x2 = \frac{-b + \sqrt{\text{delta}}}{2a}$$

Comentários, Funções para Conversão e Template Strings

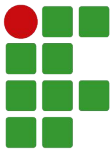
Parte 5



Sumário

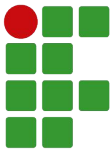
— — —

- ❏ Comentários
- ❏ Funções para conversão
- ❏ Template Strings



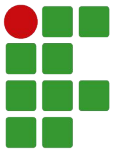
Recordando...

- ❑ Qual a diferença entre um operador e uma expressão?
- ❑ Como você escreveria em JavaScript:
***a** recebe **x** mais **y** menos 7*
Juntar as palavras “aula” e “legal”
media** recebe a média de **nota1** e **nota2



Comentários

- ❑ Comentários são trechos de nosso código que serão ignorados pelo compilador, utilizados geralmente para:
 - ❑ “Ocultar” temporariamente parte do código (estará lá, mas não será executado);
 - ❑ Incluir explicações sobre o mesmo;
 - ❑ Incluir anotações/lembretes para realizar alguma ação futura.



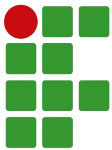
Comentários (cont.)

❏ Em JavaScript, há dois tipos de comentários:

`//Comentário em linha`

`/*Comentário em bloco`

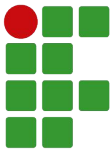
`Para uma ou mais linhas!*/`



Comentários (cont.)

- ❑ Muitas linguagens permitem usar “task tags” (tags em comentários para apontar coisas específicas a serem feitas ali):
 - ❑ `//TODO` Este comentário indica algo a ser feito
 - ❑ `//FIXME` Este outro aponta algum problema no código

No VSCode, podemos usar a extensão Todo Tree (de Gruntfuggly) para destacá-las e exibi-las em uma área separada (TODOs).



Funções para conversão

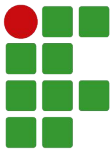
- ❑ Em JS, caso seja realizada operação entre string e outro tipo, a conversão dos dados dependerá do operador:
 - ❑ Operador suportado por string: converte não string em string;
 - ❑ Operador não suportado por string: converte string em não string.

Exemplos

"casa" + 1 //Resultado será "casa1"

true + "bola" //Resultado será "truebola"

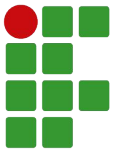
"5" * 4 //Resultado será 20



Funções para conversão (cont.)

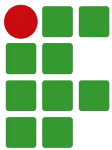
— — —

Função	Descrição	Exemplo
<code>parseInt(x)</code>	Converte string em inteiro	<code>parseInt("12.3")</code>
<code>parseFloat(x)</code>	Converte string em float (número com casa decimal)	<code>parseFloat("12.3")</code>
<code>x.toString()</code>	Converte número em string	<code>n.toString()</code> <code>(13.105).toString()</code>
<code>x.toFixed(n)</code>	Converte número em string com n casas decimais	<code>n.toFixed(2)</code> <code>(13.105).toFixed(2)</code>



Template Strings

- ❑ São strings que usam o acento grave como delimitadores (```) e apresentam uma notação específica dentro delas (`${}`) para indicar variáveis, expressões ou funções cujos resultados farão parte da string obtida;
- ❑ Também conhecidas como “strings formatadas”.



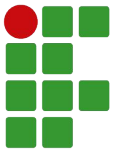
Template Strings (cont.)

- ❑ Por exemplo, se quisermos a mensagem “Olá,” juntamente com o nome de um usuário, poderíamos concatenar aquela expressão com a variável, assim:

```
"Olá, " + nome
```

- ❑ Mas, usando uma template string, podemos especificar da seguinte forma:

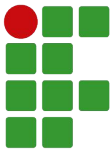
```
`Olá, ${nome}`
```



Template Strings (cont.)

- ❑ E fica ainda mais interessante quando associamos vários dados e a saída começa a ficar mais complexa:

```
`Olá, ${nome}! Seu saldo é: ${saldo}.`
```



Exercício

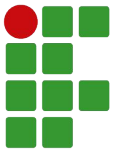
1. Crie uma página que pede o nome do usuário e suas três notas. Logo após, mostre na tela uma mensagem com o nome dessa pessoa e sua média com uma casa decimal (use uma template string para formatar a saída).

Exemplo

Olá, Christiano. Sua média é: 8.0

Comandos Condicionais

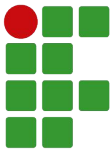
Parte 6



Sumário

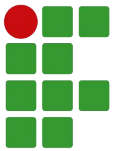
— — —

- ❏ Comandos condicionais
- ❏ Comando *if...else*
- ❏ Comando *switch*
- ❏ Operador condicional “?”



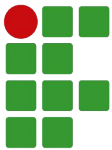
Recordando...

- ❑ Quais as duas formas de comentários que posso usar em JavaScript?
- ❑ Quais *task tags* posso usar em meu código?
- ❑ Qual função converte string em inteiro? E string em float?
- ❑ Para que serve uma string formatada? Como usar?



Comandos condicionais

- ❑ Em algumas situações, queremos que aconteçam coisas diferentes, para situações (condições) diferentes;
- ❑ Vejamos alguns exemplos...

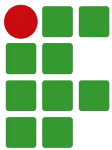


Comandos condicionais (cont.)

- ❏ Primeiro exemplo, um programa que imprime uma mensagem se um número é divisível por 2:

Leia um número N;

SE o número N é divisível por 2, **ENTÃO** escreva “{N} é divisível por 2”.



Comandos condicionais (cont.)

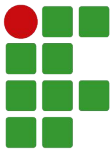
- ❑ Segundo exemplo, um programa que informa se um número é divisível por 2, por 3 ou por 5:

Leia um número N;

SE o número N é divisível por 2, **ENTÃO** escreva “{N} é divisível por 2”;

SE o número N é divisível por 3, **ENTÃO** escreva “{N} é divisível por 3”;

SE o número N é divisível por 5, **ENTÃO** escreva “{N} é divisível por 5”.



Comandos condicionais (cont.)

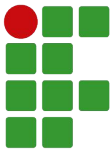
- ❑ Terceiro exemplo, um programa que informa a situação de um aluno (aprovado ou reprovado, de acordo com a média):

Leia as três notas de um aluno;

Calcule a sua média;

SE a média do aluno for superior à média mínima (6.0),
ENTÃO ele está aprovado;

SENÃO, ele está reprovado.



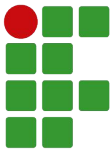
Comandos condicionais (cont.)

- ❏ Quarto exemplo, um programa que informa se um número é par ou não:

Leia um número N;

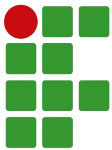
SE o número é divisível por 2, então é par;

SENÃO, ele é ímpar.



Comandos condicionais (cont.)

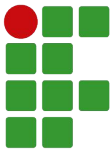
- ❑ Para tais situações, existem os comandos condicionais, que são comandos (instruções) que permitem escolher executar ou não um bloco de código de acordo com uma condição;
- ❑ Cada linguagem implementa seus próprios comandos condicionais. JavaScript apresenta três mais comuns:
 - ❑ O comando *if... else*
 - ❑ O comando *switch*
 - ❑ O comando “?”



Comando if...

- ❑ Comando presente em praticamente todas as linguagens de programação, ele permite testar se uma condição é verdadeira e, se sim, executa-se um comando ou bloco associado a ele;

- ❑ **Dica** - Para utilizar esse comando corretamente você deve saber:
 - ❑ Quando usar (ou não) um **if**;
 - ❑ Escrever corretamente a condição;
 - ❑ Quais comandos vão dentro de um **if** (e quais vão fora).



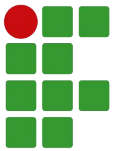
Comando if... (cont.)

Sintaxe

```
if (condição) {  
    comandos;  
}
```

Leia-se

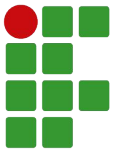
```
SE (condição for verdadeira) ENTÃO {  
    comandos;  
}
```



Comando if... - Exemplos

```
if (idade >= 18) {  
    alert("Você é maior de idade");  
}
```

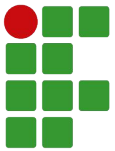
```
if (nome == "Pedro") {  
    alert("Seja bem-vindo, Pedro!");  
}
```



Comando if... - Exemplos (cont.)

```
if (aprovado) {  
    alert("Você está aprovado.");  
}
```

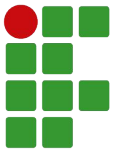
```
if (!erro) {  
    alert("Nenhum erro detectado.");  
}
```



Comando if... - Exemplos (cont.)

```
if (idade >= 18 && idade < 70) {  
    alert("Seu voto é obrigatório.");  
}
```

```
if (idade < idadeMinima || idade > idadeMaxima) {  
    alert("Idade fora da faixa etária desejada.");  
}
```



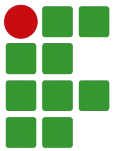
Comando if... - Exercício

1. Crie um programa um programa que imprime uma mensagem se um número é divisível por 2:

Leia um número N;

SE o número N é divisível por 2,

ENTÃO escreva “{N} é divisível por 2”.



Comando if... - Exercício (cont.)

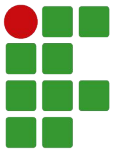
2. Crie um programa que informa se um número é divisível por 2, por 3 ou por 5:

Leia um número N;

SE o número N é divisível por 2,
ENTÃO escreva “{N} é divisível por 2”;

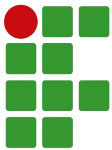
SE o número N é divisível por 3,
ENTÃO escreva “{N} é divisível por 3”;

SE o número N é divisível por 5,
ENTÃO escreva “{N} é divisível por 5”;



Comando if... else

- ❑ Em algumas situações, podemos querer executar outro bloco de código, caso a condição falhe. Nessas situações, devemos utilizar a palavra reservada **else** (“senão”) para expressar o que deve ser feito caso a condição do **if** seja falsa.



Comando if... else (cont.)

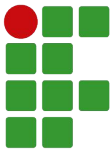
— — —

Sintaxe

```
if (condição) {  
    comandos;  
} else {  
    comandos;  
}
```

Leia-se

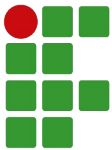
```
SE (condição for verdadeira) ENTÃO {  
    comandos;  
} SENÃO {  
    comandos;  
}
```

Comando if... else - Exemplos

```
if (idade >= 18) {  
    alert("Você é maior de idade");  
} else {  
    alert("Você é menor de idade");  
}
```

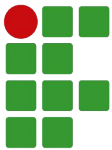
```
if (x == 5) {  
    alert("x é cinco!");  
} else {  
    alert("x não é cinco!");  
}
```



Comando if... else - Exemplos (cont.)

```
if (nome == "Pedro") {  
    alert("Seja bem-vindo, Pedro!");  
} else {  
    alert("Seja bem-vindo, senhor!");  
}
```

```
if (idade >= idadeMinima && idade <= idadeMaxima) {  
    alert("Idade dentro da faixa etária desejada.");  
} else {  
    alert("Idade fora da faixa etária desejada.");  
}
```



Comando if... else - Exercício

3. Crie um programa que informa a situação de um aluno (aprovado ou reprovado, de acordo com a média):

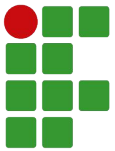
Leia as três notas de um aluno;

Calcule a sua média;

SE a média do aluno for superior à média mínima (6.0),

ENTÃO ele está aprovado;

SENÃO, ele está reprovado.



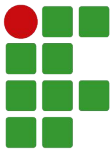
Comando if... else - Exercício (cont.)

4. Crie um programa que informa se um número é par ou não:

Leia um número N;

SE o número é divisível por 2, então é par;

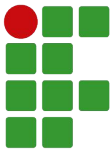
SENÃO, ele é ímpar.



Comando if... else - Observações

- ❑ Podemos ter um if sem else, mas não podemos ter um else sem if!

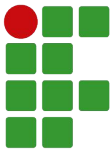
```
if (x >=18) {  
    alert("Você é maior de idade");  
} else {  
    alert("Você é menor de idade");  
} else {  
    alert("Você não pode entrar!");  
}
```



Comando if... else - Observações (cont.)

- ❑ Podemos ter ifs aninhados (um if dentro do outro).

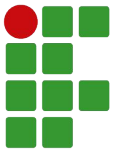
```
if (x >=18) {  
    if (genero == 'F') {  
        alert("Você já é uma mulher!");  
    } else {  
        alert("Você já é um homem!");  
    }  
}
```



Comando if... else - Observações (cont.)

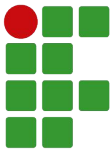
- ❑ Podemos ter ifs encadeados (um dentro do senão do outro).

```
if (nota >= 10) {  
    alert("Sua nota foi perfeita!");  
} else if (nota >= 9) {  
    alert("Sua nota foi muito boa!");  
} else if (nota >= 7.5) {  
    alert("Sua nota foi boa!");  
} else {  
    alert("É bom revisar um pouco mais.");  
}
```



Comando if... else - Exercício

5. Crie um programa que receba a idade do usuário e depois informe se o mesmo não pode votar (menos de 16 anos), pode votar (16, 17 ou 70 anos ou mais) ou deve votar (de 18 a 69 anos).



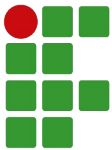
Comando switch

- ❑ Às vezes, queremos executar blocos de código de acordo com um valor escolhido ou calculado. Exemplo em um sistema:

Olá, usuário! O que você deseja?

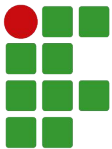
1. Cadastrar novo produto
2. Listar produtos cadastrados
3. Cadastrar novo cliente
4. Listar clientes
0. Sair

Digite sua opção:



Comando switch (cont.)

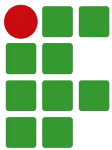
- ❑ No exemplo anterior, o código a ser executado será diferente, a depender da opção (valor numérico) escolhida pelo usuário;
- ❑ Nesta situação, dizemos que a condição vai depender de cada caso, isto é, de cada valor;
- ❑ Podemos implementar isso usando vários “if”, mas um jeito mais interessante é utilizando o comando condicional **switch**.



Comando switch (cont.)

- ❑ Permite que, a partir do valor de uma expressão (pode ser uma variável), seja executado o bloco de código associado àquele caso (valor);
- ❑ Sua sintaxe é:

```
switch (expressão) {  
    case valor1:  
        comandos;  
        break;  
    case valor2:  
        comandos;  
        break;  
    case valorN:  
        comandos;  
        break;  
    default:  
        comandos;  
}
```

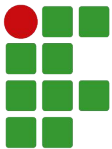


Comando switch (cont.)

Onde:

- ❑ **switch** - palavra reservada que informa a expressão a ser calculada e, em seguida, os casos (rótulos) para o sistema testar;
- ❑ **expressão** - aqui vai a expressão a ser avaliada (isto é, calculada);
- ❑ **case** - informa o valor que, caso coincida com o resultado da expressão, os comandos presentes daquele ponto até o final do switch serão executados;

```
switch (expressão) {  
    case valor1:  
        comandos;  
        break;  
    case valor2:  
        comandos;  
        break;  
    case valorN:  
        comandos;  
        break;  
    default:  
        comandos;  
}
```

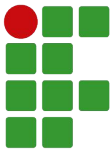


Comando switch (cont.)

Onde: (cont.)

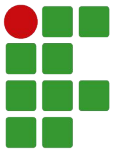
- ❑ **break** - realiza uma “quebra” na execução do switch, “pulando” para logo após o mesmo. Geralmente aparece após a lista de comandos de um case;
- ❑ **default** - especifica uma série de comandos a serem executados, caso o resultado da expressão não “case” com nenhum dos valores passados. É opcional.

```
switch (expressão) {  
    case valor1:  
        comandos;  
        break;  
    case valor2:  
        comandos;  
        break;  
    case valorN:  
        comandos;  
        break;  
    default:  
        comandos;  
}
```



Comando switch - Exemplo

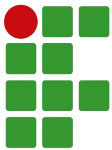
- ❑ Vamos criar uma calculadora de áreas para figuras geométricas!
- ❑ O usuário inicialmente escolherá (informando o número da opção) uma das seguintes figuras:
 1. Triângulo ($\text{area} = \text{base} * \text{altura} / 2$);
 2. Quadrado ($\text{area} = \text{lado} * \text{lado}$);
 3. Círculo ($\text{area} = \text{Math.PI} * \text{raio} * \text{raio}$).
- ❑ Depois disso, vamos pedir as medidas da figura geométrica, calcular sua área e exibir;
- ❑ Caso ele informe uma opção inválida, informe isso.



Comando switch - Exercício

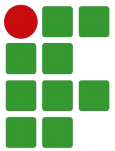
6. Crie um programa que permita ao usuário informar uma das seguintes conversões:
 1. KB para MB (divide por 1024);
 2. KB para GB (divide por (1024×1024));
 3. MB para KB (multiplica por 1024);
 4. GB para KB (multiplica por (1024×1024)).

Em seguida, o usuário informará o valor na medida original e o sistema exibirá o valor correspondente na medida alvo.



Operador condicional

- ❑ Há situações em que, caso uma condição seja verdadeira, queremos retornar um valor, caso contrário, outro;
- ❑ Podemos alcançar esse objetivo por meio da instrução `if...else`, mas também podemos fazê-lo por meio do operador condicional “?”, também conhecido como operador ternário.



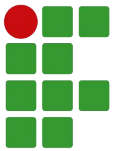
Operador condicional (cont.)

Sintaxe

condição ? valor_se_verdadeiro : valor_se_falso

Exemplo 1

```
var a = (b > 0 ? b : 0);
```



Operador condicional (cont.)

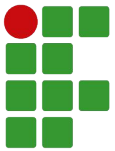
Exemplo 2

Um programa que receba um número e retorne o dobro se ele for positivo, caso contrário retorne a metade:

```
var n = parseFloat( prompt("Informe o valor de n") );  
var resposta = (n > 0 ? 2*n : n/2);  
alert("Resposta: " + n);
```

Comandos de Repetição

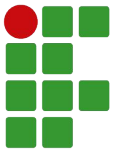
Parte 7



Sumário

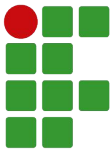
— — —

- ❏ Comandos de repetição
- ❏ Comando *for*
- ❏ Comando *while*
- ❏ Comando *do...while*
- ❏ Comandos para interrupção de laços



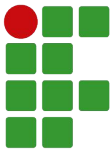
Recordando...

- ❑ Para que serve um comando condicional?
- ❑ Quais as três opções de comandos condicionais que estudamos? Cite um exemplo com cada uma delas.



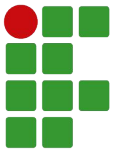
Comandos de repetição

- ❑ Para resolver alguns problemas, precisamos repetir um conjunto de instruções. Exemplos:
 - ❑ Imprimir todos os números de 1 a 100;
 - ❑ Imprimir somente os números pares de 1 a 100;
 - ❑ Imprimir a soma de todos os números de 1 a 100;
 - ❑ Receber dois números inteiros e imprimir todos os inteiros do maior até o menor deles.



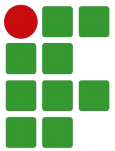
Comandos de repetição

- ❑ Para cada uma dessas situações, queremos executar uma série de passos (instruções) para cada número daquele intervalo, a fim de encontrarmos a solução;
- ❑ Assim, precisamos de comandos de repetição, instruções que permitem repetir um bloco de código sob determinadas circunstâncias;
- ❑ JavaScript oferece três instruções para repetição: *for*, *while* e *do...while*.



Comando for

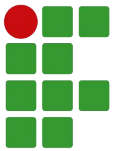
- ❏ Este comando de repetição permite especificar uma variável de controle que será usada para “varrer” os números de um intervalo (de 5 até 10, por exemplo) e como essa variável deve ser incrementada/decrementada ao final de cada passo.



Comando for

Sintaxe

```
for (inicialização; condição; incremento) {  
    comandos;  
}
```



Comando for



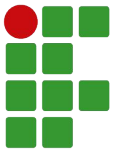
Exemplo 1: Imprimir todos os números de 1 a 100

Código JavaScript

```
for (let i = 1; i <= 100; i++) {  
    alert(i);  
}
```

Leia-se

PARA $i = 1$; ENQUANTO $i \leq 100$ FAÇA {
 MOSTRE i ;
} (após cada iteração, aumente i em 1)



Comando for



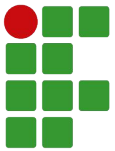
Exemplo 2a: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
for (let i = 1; i <= 100; i++) {  
    if (i % 2 == 0) {  
        alert(i);  
    }  
}
```

Leia-se

```
PARA i = 1; ENQUANTO i <= 100 FAÇA {  
    SE (resto de i por 2 == 0) ENTÃO {  
        MOSTRE i;  
    }  
} (após cada iteração, aumente i em 1)
```



Comando for



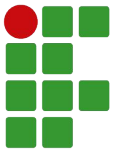
Exemplo 2b: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
for (let i = 2; i <= 100; i+=2) {  
    alert(i);  
}
```

Leia-se

PARA $i = 2$; ENQUANTO $i \leq 100$ FAÇA {
 MOSTRE i ;
} (após cada iteração, aumente i em 2)



Comando for



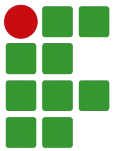
Exemplo 3: Imprimir soma de todos os números de 1 a 100

Código JavaScript

```
let soma = 0;
for (let i = 1; i <= 100; i++) {
    soma = soma + i;
}
alert(soma);
```

Leia-se

```
soma = 0;
PARA i = 1; ENQUANTO i <= 100 FAÇA {
    soma = soma + i;
} (após cada iteração, aumente i em 1)
MOSTRE soma;
```



Comando for

— — —



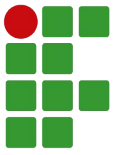
Exemplo 4: Receber dois números inteiros e imprimir todos os inteiros do maior até o menor deles

Código JavaScript

```
let a = parseInt(prompt("Informe a"));
let b = parseInt(prompt("Informe b"));
let maior = (a > b ? a : b);
let menor = (a < b ? a : b);
for (let x = maior; x >= menor; x--) {
    alert(x);
}
```

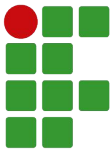
Leia-se

```
LEIA a;
LEIA b;
maior = maior valor entre a e b;
menor = menor valor entre a e b;
PARA x = maior; ENQUANTO x >= menor FAÇA {
    MOSTRE x;
} (após cada iteração, diminua x em 1)
```



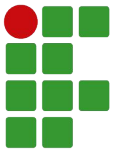
Comando for - Exercício

1. Escreva um programa que recebe dois números inteiros e informe a soma de todos os números daquele intervalo;
2. Escreva um programa que recebe um número inteiro e informe o seu fatorial, sendo que:
 - a. Fatorial de números negativos não existe
 - b. Fatorial de 0 é 1
 - c. Fatorial de n é $1*2*3*...*n$



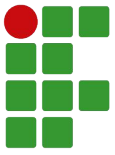
Comando for - Exercício

3. Escreva um programa que recebe 10 números e, ao final, informe o maior deles;
4. Escreva um programa que recebe 10 números inteiros e, ao final, informe quantos desses números eram pares e quantos eram ímpares;



Comando for - Exercício

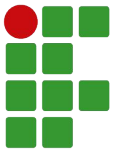
5. Escreva um programa que recebe um número inteiro positivo e, em seguida, mostre todos os divisores desse número;
6. Escreva um programa que recebe um número inteiro positivo e, em seguida, informa se esse número é primo ou não.



Comando while

— — —

- ❏ Este comando de repetição permite especificar uma condição de controle e, enquanto for verdadeira, repetirá o bloco de código vinculado.

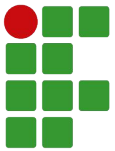


Comando while

— — —

Sintaxe

```
while (condição) {  
    comandos;  
}
```



Comando while



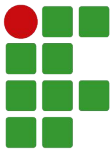
Exemplo 1: Imprimir todos os números de 1 a 100

Código JavaScript

```
let i = 1;
while (i <= 100) {
    alert(i);
    i++;
}
```

Leia-se

```
i = 1;
ENQUANTO i <= 100 FAÇA {
    MOSTRE i;
    INCREMENTE i em 1;
}
```



Comando while



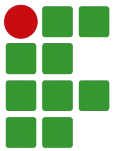
Exemplo 2a: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
let i = 1;
while (i <= 100) {
    if (i % 2 == 0) {
        alert(i);
    }
    i++;
}
```

Leia-se

```
i = 1;
ENQUANTO i <= 100 FAÇA {
    SE (resto de i por 2 == 0) ENTÃO {
        MOSTRE i;
    }
    INCREMENTE i em 1;
}
```



Comando while



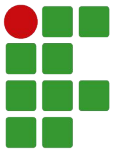
Exemplo 2b: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
let i = 2;
while (i <= 100) {
    alert(i);
    i+=2;
}
```

Leia-se

```
i = 2;
ENQUANTO i <= 100 FAÇA {
    MOSTRE i;
    INCREMENTE i em 2;
}
```



Comando while



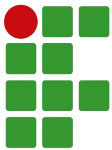
Exemplo 3: Imprimir soma de todos os números de 1 a 100

Código JavaScript

```
let soma = 0;
let i = 1;
while (i <= 100) {
    soma = soma + i;
    i++;
}
alert(soma);
```

Leia-se

```
soma = 0;
i = 1;
ENQUANTO i <= 100 FAÇA {
    soma = soma + i;
    INCREMENTE i em 1;
}
MOSTRE soma;
```



Comando while

— — —

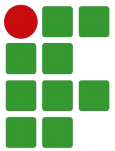
Exemplo 4: Receber dois números inteiros e imprimir todos os inteiros do maior até o menor deles

Código JavaScript

```
let a = parseInt(prompt("Informe a"));
let b = parseInt(prompt("Informe b"));
let maior = (a > b ? a : b);
let menor = (a < b ? a : b);
let x = maior;
while (x >= menor) {
    alert(x);
    x--;
}
```

Leia-se

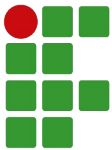
```
LEIA a;
LEIA b;
maior = maior valor entre a e b;
menor = menor valor entre a e b;
x = maior;
ENQUANTO x >= menor FAÇA {
    MOSTRE x;
    DECREMENTE x em 1;
}
```

Comando while - Exercício

7. Escreva um programa que recebe um número inteiro e imprime na tela o seu fatorial, sendo que:
 - a. Fatorial de números negativos não existe
 - b. Fatorial de 0 = 1
 - c. Fatorial de $n = 1*2*3*...*n$

8. Escreva um programa que calcule a soma de todos os números positivos recebidos. Quando o usuário informar um número negativo ou zero, mostre a soma encontrada e encerre o programa.



Comando while - Exercício

9. Escreva um programa para votação que apresente a seguinte tela:

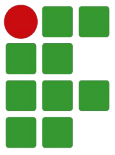
Vote em sua fruta preferida:

1. Maçã

2. Pera

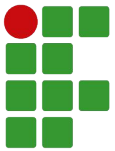
0. Encerrar votação

Caso o usuário vote 1 ou 2, contabilize o voto e mostre uma mensagem informando a opção escolhida pelo usuário. Repita o processo até o usuário encerrar a votação (escolhendo 0), quando deve exibir o total de votos em cada fruta e a vencedora.



Comando `do...while`

- ❑ Em certas situações, queremos que o bloco de código a ser repetido seja executado antes de testar a condição de controle, obrigando assim aquele bloco a ser executado pelo menos uma vez;
- ❑ Nessas circunstâncias, podemos utilizar a instrução `do...while`, que trabalha de forma similar ao `while`, porém testando a condição de controle somente após executar o bloco.

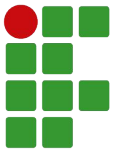


Comando do...while

— — —

Sintaxe

```
do {  
    comandos;  
} while (condição);
```



Comando do...while

— — —

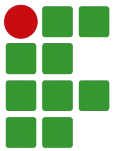
Exemplo 1: Imprimir todos os números de 1 a 100

Código JavaScript

```
let i = 1;  
do {  
    alert(i);  
    i++;  
} while (i <= 100);
```

Leia-se

```
i = 1;  
FAÇA {  
    MOSTRE i;  
    INCREMENTE i em 1;  
} ENQUANTO i <= 100;
```



Comando do...while

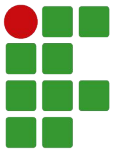
Exemplo 2a: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
let i = 1;
do {
    if (i % 2 == 0) {
        alert(i);
    }
    i++;
} while (i <= 100);
```

Leia-se

```
i = 1;
FAÇA {
    SE (resto de i por 2 == 0) ENTÃO {
        MOSTRE i;
    }
    INCREMENTE i em 1;
} ENQUANTO i <= 100;
```



Comando do...while

— — —

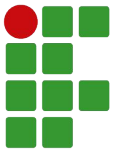
Exemplo 2b: Imprimir todos os números pares de 1 a 100

Código JavaScript

```
let i = 2;  
do {  
    alert(i);  
    i+=2;  
} while (i <= 100);
```

Leia-se

```
i = 2;  
FAÇA {  
    MOSTRE i;  
    INCREMENTE i em 2;  
} ENQUANTO i <= 100;
```



Comando do...while

— — —

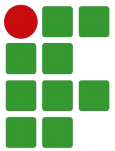
Exemplo 3: Imprimir soma de todos os números de 1 a 100

Código JavaScript

```
let soma = 0;
let i = 1;
do {
    soma = soma + i;
    i++;
} while (i <= 100);
alert(soma);
```

Leia-se

```
soma = 0;
i = 1;
FAÇA {
    soma = soma + i;
    INCREMENTE i em 1;
} ENQUANTO i <= 100;
MOSTRE soma;
```

Comando do...while

— — —

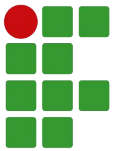
Exemplo 4: Receber dois números inteiros e imprimir todos os inteiros do maior até o menor deles

Código JavaScript

```
let a = parseInt(prompt("Informe a"));
let b = parseInt(prompt("Informe b"));
let maior = (a > b ? a : b);
let menor = (a < b ? a : b);
let x = maior;
do {
    alert(x);
    x--;
} while (x >= menor);
```

Leia-se

```
LEIA a;
LEIA b;
maior = maior valor entre a e b;
menor = menor valor entre a e b;
x = maior;
FAÇA {
    MOSTRE x;
    DECREMENTE x em 1;
} ENQUANTO x >= menor;
```



Comparando *while* e *do...while*

— — —

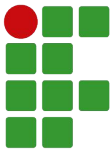
Nos casos abaixo, o que acontecerá se o valor passado para “a” for...
5? 1? 0? -5?

Código usando while

```
let a = parseInt(prompt("Informe a"));  
while (a > 0) {  
    alert(a);  
    a--;  
}
```

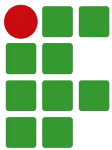
Código usando do...while

```
let a = parseInt(prompt("Informe a"));  
do {  
    alert(a);  
    a--;  
} while (a > 0);
```



Comando do...while - Exercício

10. Escreva um programa que recebe um número inteiro e imprime na tela o seu fatorial, sendo que:
 - a. Fatorial de números negativos não existe
 - b. Fatorial de 0 = 1
 - c. Fatorial de $n = 1*2*3*...*n$
11. Escreva um programa que calcule a soma de todos os números positivos recebidos. Quando o usuário informar um número negativo ou zero, mostre a soma encontrada e encerre o programa.



Comando do...while - Exercício

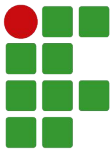
— — —

12. Crie um programa que permita ao usuário informar uma das seguintes opções:

1. *KB para MB;*
2. *KB para GB;*
3. *MB para KB;*
4. *GB para KB;*
0. *Sair.*

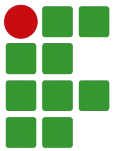
Caso o usuário escolha uma opção de 1 a 4, ele informará o valor na medida original e o sistema informará o valor correspondente na medida alvo. Caso informe uma opção inválida (5, por exemplo), o sistema emitirá mensagem dizendo que a opção escolhida é inválida. Em ambos os casos, após processar a opção escolhida, o programa imprimirá novamente as opções e o usuário poderá informar nova opção.

O programa somente encerrará quando o usuário informar a opção “0” (sair).



Comandos para interrupção de laços (repetições)

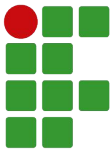
- ❑ Em algumas situações, podemos querer interromper um comando de repetição antes que ele alcance a condição de parada, por exemplo quando encontra um valor esperado;
- ❑ Nessas situações, temos duas opções de instrução para interrupção do laço: **break** e **continue**.



Comando break

— — —

- ❏ Quando executado, interrompe a execução dos comandos dentro do laço (*for*, *while* ou *do...while*) ou *switch* e sai imediatamente do mesmo.



Comando break

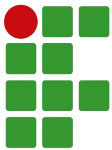
Exemplo

```
for (let i = 1; i <= 10; i++) {  
    if (i % 3 == 0) {  
        break;  
    }  
    alert(i + " não é divisível por 3");  
}  
alert("===Programa encerrado===");
```



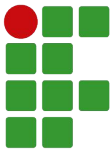
Saída

```
1 não é divisível por 3  
2 não é divisível por 3  
===Programa encerrado===
```



Comando continue

- ❑ Quando executado, interrompe a execução dos comandos dentro do laço (*for*, *while* ou *do...while*), indo para o final do mesmo, executando assim a seção de incremento/decremento (caso o comando seja um *for*) e testando novamente a condição para continuar repetindo.



Comando continue

Exemplo

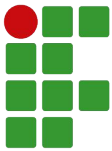
```
for (let i = 1; i <= 10; i++) {  
    if (i % 3 == 0) {  
        continue;  
    }  
    alert(i + " não é divisível por 3");  
}  
alert("===Programa encerrado===");
```

7.17



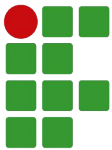
Saída

```
1 não é divisível por 3  
2 não é divisível por 3  
4 não é divisível por 3  
5 não é divisível por 3  
7 não é divisível por 3  
8 não é divisível por 3  
10 não é divisível por 3  
===Programa encerrado===
```



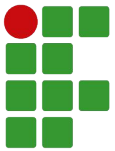
Quando usar `break` ou `continue`?

- ❑ Em comandos *switch*, use *break*;
- ❑ Em comandos de repetição, use *break* se, naquele ponto, desejar interromper completamente a execução do laço e seguir com o programa...
- ❑ ...ou use *continue*, caso queira interromper somente a execução atual, “pulando” para a próxima.



Comando break / continue - Exercício

13. Crie um programa que recebe dois números inteiros positivos e, em seguida, exibe o MDC (máximo divisor comum) deles;
14. Crie um programa que recebe dois números inteiros positivos e, em seguida, exibe o MMC (mínimo múltiplo comum) deles.



Comando break / continue - Exercício

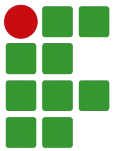
15. Jogo “Acerte o Número”! Crie um programa que gera um número secreto (um inteiro aleatório de 0 a 49) e, em seguida, permite ao jogador informar um número. O programa, então, informa se o número secreto é maior ou menor que o número escolhido e o jogo continua. Caso o jogador acerte, ele deve ser parabenizado, informado quanto ao total de tentativas e o jogo está encerrado.

Dica - use a seguinte linha para gerar o número aleatório:

```
let secreto = parseInt(Math.random()*50);
```

Funções e Eventos

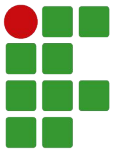
Parte 8



Sumário

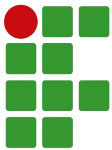
— — —

- ❏ Declarando e Executando Funções
- ❏ Eventos em JavaScript



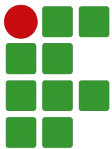
Recordando...

- ❑ Para que serve um comando de repetição?
- ❑ Quais as três opções de comandos de repetição que estudamos? Qual a diferença entre eles?



Funções

- ❑ Uma função é uma rotina (conjunto de instruções), geralmente composta por um cabeçalho (assinatura, que inclui seu nome e argumentos) e um corpo;
- ❑ De forma similar a variáveis, uma função deve ser declarada antes de ser acessada (executada);
- ❑ Ao executar uma função, deve-se tomar o cuidado de sempre incluir os parênteses, mesmo se a função não tiver argumentos.



Funções

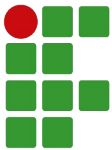
— — —

Sintaxe (declarando uma função)

```
function nomeFuncao(argumentos) {  
    comandos;  
}
```

Sintaxe (executando uma função)

```
nomeFuncao(argumentos);
```



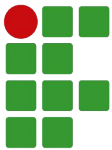
Exemplos de declarações

//Exibe um alert cumprimentando - Declarando

```
function cumprimentar() {  
    let nome = prompt("Qual é o seu nome?");  
    alert(`Olá, ${nome}!`);  
}
```

//Executando

```
cumprimentar();
```



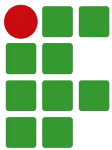
Exemplos de declarações

//Muda cor de fundo da página - Declarando

```
function mudarCor(cor) {  
    document.backgroundColor = cor;  
}
```

//Executando

```
mudarCor("red");
```



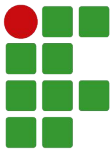
Exemplos de declarações

//Retorna a média de dois números - Declarando

```
function calcularMedia(a, b) {  
    return (a + b) / 2;  
}
```

//Executando

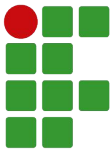
```
alert( calcularMedia(5, 7) );
```



Eventos em JavaScript

- ❑ Nos navegadores, podemos querer que algumas ações aconteçam como consequência de alguma interação com o usuário. Exemplos:
 - ❑ Ao carregar uma página (onload);
 - ❑ Ao clicar em algum elemento (onclick);
 - ❑ Ao mudar o valor de algum elemento (onchange);
 - ❑ Ao selecionar um campo em um formulário (onfocus).

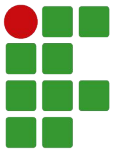
- ❑ Tais interações geralmente são conhecidas como eventos e podemos executar código em JavaScript a partir delas.



Eventos em JavaScript

Exemplo 1

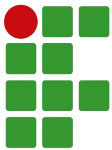
```
<button onclick="cumprimentar()">Olá!</button>
<script>
  function cumprimentar() {
    let nome = prompt("Qual é o seu nome?");
    alert(`Olá, ${nome}!`);
  }
</script>
```



Eventos em JavaScript

Exemplo 2

```
<input id="entrada">
<button onclick="dobrar()">Dobrar</button>
<input id="saida">
<script>
    function dobrar() {
        let n = parseFloat(entrada.value);
        saida.value = 2*n;
    }
</script>
```

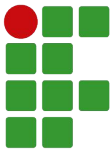


Eventos em JavaScript

Exemplo 3

```
<input id="entrada1">
<input id="entrada2">
<button id="btnCalcular">Calcular</button>
<input id="saida">
<script>
    function calcularMedia() {
        let a = parseFloat(entrada1.value);
        let b = parseFloat(entrada2.value);
        saida.value = (a+b)/2;
    }

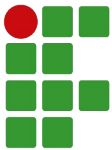
    //Aqui, apontamos diretamente para a função, sem os parênteses
    btnCalcular.onclick = calcularMedia;
</script>
```

Eventos em JavaScript

Exemplo 4

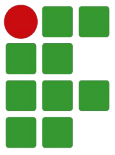
```
<button onclick="alterarQuantidade(-1)">-</button>
<input id="saida">
<button onclick="alterarQuantidade(+1)">+</button>
<script>
  let quantidade = 0;
  function alterarQuantidade(dif) {
    quantidade += dif;
    saida.value = quantidade;
  }
  alterarQuantidade(0);
</script>
```



Eventos em JavaScript

Exemplo 5

```
<input id="entrada" onchange="multiplicar()">
<select id="fator" onchange="multiplicar()">
  <option value="1">x1</option>
  <option value="2">x2</option>
  <option value="3">x3</option>
</select>
<input id="saida">
<script>
  function multiplicar() {
    saida.value = Number(entrada.value)*Number(fator.value);
  }
</script>
```

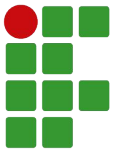


Exercício

1. Escreva um programa em que o usuário possa informar valores em um `<input>` e, ao clicar em `<button>`, apareça a soma desses valores em outro `<input>`;
2. Escreva um programa em que o usuário possa informar um valor em um `<input>` e, ao clicar em um `<button>`, informe em outro `<input>` se aquele número é primo ou não;
3. Escreva um programa que, dados valor de depósito mensal, taxa de juros mensal e quantidade de meses (informados por meio de três `<input>`), ao clicar em um `<button>`, calcule e mostre o montante final da aplicação.

Objetos e Arrays

Parte 9



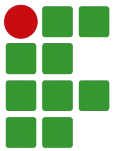
Sumário

— — —

❏ Objetos

❏ Arrays

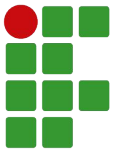
❏ Comando `for...in`



Recordando...

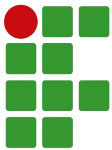
— — —

- ☐ Para que serve uma função?
- ☐ Para que serve um evento?



Objetos

- ❑ Trata-se de um variável composta (isto é, que pode guardar vários valores) cujos valores podem ser armazenados ou recuperados a partir de nomes (propriedades);
- ❑ Em JavaScript, a quantidade de propriedades em um objeto é variável, isto é, podemos alterar ou acrescentar novas propriedades a qualquer momento.



Declarando objetos

Sintaxe #1

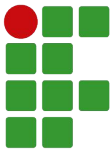
```
variavel = new Object( valor ); //Valor é opcional
```

Exemplo

```
aluno = new Object( "Joaquim" );
```

```
carro = new Object();
```

```
diario = new Object;    //Não recomendado!
```

Declarando objetos

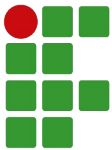


Sintaxe #2

```
variavel = { propriedade: valor, ... }; //Recomendado!
```

Exemplo

```
aluno = {matricula: 1, nome: "Joaquim", anoNascimento: 1987};  
carro = {};
```



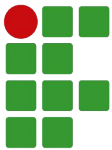
Atribuindo valores às propriedades

Sintaxe #1

```
variavel.propriedade = valor; //Recomendado
```

Exemplo

```
aluno.nome = "Zacarias";
```



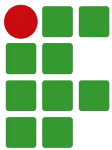
Atribuindo valores às propriedades

Sintaxe #2

```
variavel["propriedade"] = valor;
```

Exemplo

```
aluno["nome"] = "Zacarias";
```



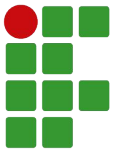
Definindo métodos

Sintaxe #1

```
variavel.metodo = function () {...}; //Recomendado
```

Exemplo

```
aluno.pegarIdade = function () {  
    return 2022 - this.anoNascimento;  
};
```



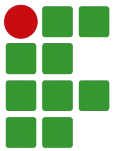
Definindo métodos

Sintaxe #2

```
variavel["metodo"] = function () {...};
```

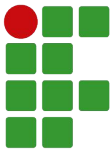
Exemplo

```
aluno["pegarIdade"] = function () {  
    return 2022 - this.anoNascimento;  
};
```



Vetores ou arrays

- ❑ Trata-se de um variável composta cujos valores podem ser armazenados ou recuperados a partir de um índice;
- ❑ Em JavaScript, a quantidade de elementos em um array é variável, isto é, podemos acrescentar novos elementos a qualquer momento;
- ❑ O índice de um array geralmente é numérico mas também pode ser textual (array associativo).



Declarando arrays

Sintaxe #1

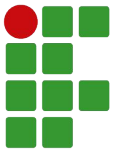
```
variavel = new Array(tamanho); //Tamanho é opcional
```

Exemplo

```
diasDaSemana = new Array(7);
```

```
meses = new Array();
```

```
anos = new Array; //Não recomendado!
```



Declarando arrays

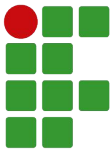


Sintaxe #2

```
variavel = [elemento1, elemento2... ];
```

Exemplo

```
diasUteis = ["Segunda", "Terça", "Quarta", "Quinta", "Sexta"];  
meses = [];
```

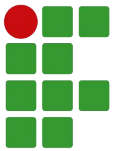
Atribuindo valores ao array

Sintaxe

```
variavel[indice] = valor;
```

Exemplo

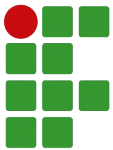
```
meses[0] = "Janeiro";  
notas["Joaquim"] = 8.0;  
notas["Cláudia"] = [8.0, 9.0];
```



Propriedades de um array

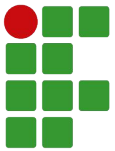
— — —

Propriedade	Descrição	Exemplo
length	Retorna o tamanho do array	meses.length



Métodos de um array

Método	Descrição	Exemplo
indexOf(elemento)	Retorna a posição do elemento dentro do array	meses.indexOf("Janeiro")
push(elementos)	Adiciona novos elementos ao final do array, retornando o novo tamanho	meses.push("Fevereiro", "Março")
pop()	Remove e retorna o último elemento do array	meses.pop()
unshift(elementos)	Adiciona novos elementos ao início do array, retornando o novo tamanho	meses.unshift("Janeiro")
shift()	Remove e retorna o primeiro elemento do array	meses.shift()
toString()	Retorna o array como string	meses.toString()



Comando for...in

— — —

- ❑ Trata-se de um comando de repetição que, para cada propriedade de um objeto ou elemento de um array, executará o comando associado.

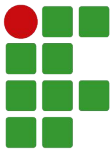
Sintaxe

```
for (let propriedade in objeto) {  
    comandos;  
}
```

Onde:

propriedade – string com nome da propriedade;

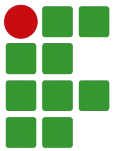
objeto[propriedade] – valor da propriedade do objeto/array.



Comando for...in

Exemplo 1

```
let aluno = {  
  matricula: 123,  
  nome: "Christiano",  
  idade: 37  
};  
  
for (let prop in aluno) {  
  alert(`${prop} -> ${aluno[prop]}`);  
}
```

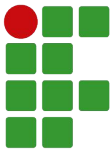


Comando for...in



Exemplo 2

```
let meses = [  
    "Janeiro",  
    "Fevereiro",  
    "Março"  
];  
  
for (let prop in meses) {  
    alert(`${prop} -> ${meses[prop]}`);  
}
```



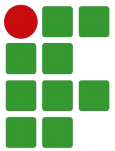
Comando for...in

Exemplo 3

```
let dicionario = [];  
dicionario["casa"] = "house";  
dicionario["amarelo"] = "yellow";  
dicionario[10] = 1;  
  
for (let prop in dicionario) {  
    alert(`${prop} -> ${dicionario[prop]}`);  
}
```

Date, Math e String

Parte 10



Sumário

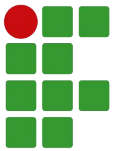
— — —

📄 Objetos globais

📄 Date

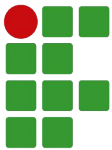
📄 Math

📄 String



Recordando...

- ❑ O que é um objeto? Para que serve um objeto? Dê um exemplo de objeto em JavaScript;
- ❑ O que é um array? Para que serve um array? Dê um exemplo de array em JavaScript.

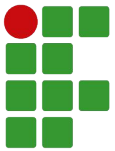


Objetos globais

- ❑ Em JavaScript, são objetos presentes no escopo global, isto é, podem ser acessados a partir de qualquer lugar no código;
- ❑ Mantendo tais objetos no escopo global, podemos acessar tais objetos e suas propriedades e métodos em qualquer lugar;
- ❑ Alguns objetos padrões do JavaScript com tal característica são o Date, Math e String.

Saiba mais em:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects



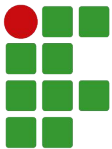
Date

- ❑ Permite armazenar objetos do tipo Date (data e hora) em variáveis;

Exemplos

```
//Cria um objeto com a data e hora atual  
hoje = new Date();
```

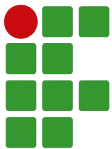
```
//Ano, mês, dia, horas, minutos, segundos  
natal = new Date(2017, 11, 25, 5, 0, 0);
```



Métodos de Date

— — —

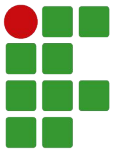
Método	Descrição	Exemplo
getDate()	Retorna o dia do mês	hoje.getDate()
getMonth()	Retorna o mês (0 para janeiro)	hoje.getMonth()
getFullYear()	Retorna o ano, com 04 dígitos	hoje.getFullYear()
getHours()	Retorna as horas	hoje.getHours()
getMinutes()	Retorna os minutos	hoje.getMinutes()
getSeconds()	Retorna os segundos	hoje.getSeconds()
getDay()	Retorna o dia da semana (0 para domingo)	hoje.getDay()
toString()	Retorna a data como string	hoje.toString()



Métodos de Date

— — —

Método	Descrição	Exemplo
setDate(x)	Altera o dia do mês	hoje.setDate(10)
setMonth(x)	Altera o mês	hoje.setMonth(1)
setFullYear(x)	Altera o ano	hoje.setFullYear(2000)
setHours(x)	Altera as horas	hoje.setHours(12)
setMinutes(x)	Altera os minutos	hoje.setMinutes(40)
setSeconds(x)	Altera os segundos	hoje.setSeconds(55)



Math

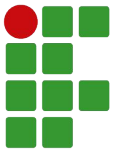
- ❑ Possui várias propriedades e métodos úteis para cálculos matemáticos;

Exemplos

```
//Retorna o maior dentre uma série de números  
maior = Math.max(7, 12, 8);
```

```
//Retorna um número aleatório entre 0 e 1  
aleatorio = Math.random();
```

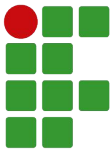
```
//Retorna o valor do número arredondado  
valor = Math.round( 5.8 );
```



Propriedades de Math

— — —

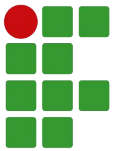
Propriedade	Descrição	Exemplo
E	Retorna a constante de Euler (aprox. 2,718)	Math.E
PI	Retorna a constante PI (aprox. 3,14)	Math.PI



Métodos de Math

— — —

Método	Descrição	Exemplo
abs(x)	Retorna o valor absoluto de um número	Math.abs(-1)
ceil(x)	Retorna o número inteiro arredondado para cima	Math.ceil(5.5)
floor(x)	Retorna o número inteiro arredondado para baixo	Math.floor(5.5)
max(numeros)	Retorna o maior número de uma série	Math.max(5, 8, 9)
min(numeros)	Retorna o menor número de uma série	Math.min(5, 8, 9)
random()	Retorna um número aleatório decimal entre 0 e 1	Math.random()
round(x)	Retorna o valor do número arredondado	Math.round(5.5)



String

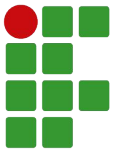
- ❑ Possui várias propriedades e métodos úteis para manipulação de strings;

Exemplos

```
//Retorna o tamanho da string  
tamanho = nome.length;
```

```
//Retorna a posição de uma substring dentro da string  
posicao = nome.indexOf("Silva");
```

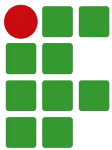
```
//Retorna a string em letras maiúsculas  
caixaAlta = nome.toUpperCase();
```



Propriedades de String

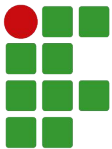
— — —

Propriedade	Descrição	Exemplo
length	Retorna o tamanho da string	nome.length



Métodos de String

Método	Descrição	Exemplo
charAt(indice)	Retorna o caracter na posição especificada	nome.charAt(0)
indexOf(substring)	Retorna a posição de substring dentro da string	nome.indexOf("Silva")
replace(antiga, nova)	Retorna uma nova string substituindo a substring antiga pela nova na string passada	nome.replace("Silva", "Souza")
slice(inicio, fim)	Retorna uma substring, começando na posição inicio e terminando na posição fim (mas não incluindo esta)	nome.slice(5, 8)
split(separador)	“Quebra” a String baseada no caracter separador, retornando-a como um array de substrings	nome.split(" ")
toLowerCase()	Retorna a string em letras minúsculas	nome.toLowerCase()
toUpperCase()	Retorna a string em letras maiúsculas	nome.toUpperCase()



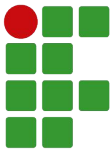
Exercício

1. Vamos criar uma página que apresente o dia da semana, a data e a hora no formato:

Dia da semana, dia de mês de ano, hora:minuto

Exemplo:

Sábado, 25 de outubro de 2022, 17:00



Exercício

2. Crie uma página que receba o nome completo de uma pessoa em um campo de texto e, após clicar em um botão, exibe seu nome segundo a norma ABNT, isto é:
 - a. Último nome em letras maiúsculas;
 - b. Se houver mais de um nome, primeiro nome começando por uma letra maiúscula e separado do anterior por uma vírgula;
 - c. Demais nomes do meio, somente a inicial em maiúscula, seguida por ponto;
 - d. E um ponto final, se necessário.

Exemplos:

SANTOS.

SANTOS, Christiano.

SANTOS, Christiano L.

SANTOS, Christiano F. L.

E... Acabou!
Boas Férias (quase lá)!